

Jan Andrzejewski

AI-first Developer · Student Automatyki i Robotyki

early adopter LLM, RAG i agentów

janekandr@gmail.com · Polska · GitHub · LinkedIn

Podsumowanie

Inżynier AI / Full-Stack Developer specjalizujący się w systemach agentycznych i architekturze systemów produkcyjnych. Łączę solidne fundamenty inżynierskie (embedded, automatyka) z nowoczesnym stackiem webowym oraz doświadczeniem w budowaniu pipeline'ów RAG. Early adopter narzędzi AI-assisted development (od Copilota po Claude Code i AntigraVity 2.0). **W implementacjach kładę nacisk na świadome decyzje architektoniczne, optymalizację tokenów i wydajność infrastruktury, co pozwala na realne cięcie kosztów utrzymania systemów AI.** Student Automatyki i Robotyki realizujący autorskie projekty produktowe end-to-end.

Doświadczenie

PSI Polska

psi.pl

(stanowisko i okres do uzupełnienia)

Projekty

DocBase — RAG SaaS „drugi mózg” dla MŚP 2026 — własny projekt produktowy (2-osobowy zespół) · docbase.pl

Problem. W polskich MŚP wiedza operacyjna jest rozproszona po mailach, PDF-ach, dokumentach i głowach kluczowych pracowników. Onboarding nowej osoby zajmuje tygodnie, a odejście doświadczonego pracownika to realne ryzyko ciągłości biznesu.

Rozwiązanie. Wielonajemcza platforma SaaS z pipeline'em RAG, która indeksuje firmowe dokumenty i odpowiada na pytania w języku naturalnym, podając link do źródła. Zaufanie buduje weryfikowalność odpowiedzi i ścisła izolacja danych każdej firmy na poziomie bazy.

Wartość biznesowa. Skrócenie onboardingu, oszczędność czasu na szukaniu procedur, zabezpieczenie wiedzy firmowej. Architektura przygotowana pod opcjonalny wariant on-premise dla klientów z restrykcyjnym RODO.

Decyzje kosztowe i optymalizacje:

- Routing modeli przez **LiteLLM** (tańszy model do prostych zapytań, mocniejszy do analiz), tani model embeddingowy, świadome cięcie tokenów.
- Zastąpienie dedykowanej **bazy wektorowej** zoptymalizowaną tabelą **PostgreSQL**, redukując koszty infrastruktury o rzędy wielkości. Przeprowadziłem trade-off między natychmiastowym czasem odpowiedzi a kosztami RAM-u, dopasowując architekturę do faktycznych wymagań biznesowych.

Stack: Python + **FastAPI**, **LlamaIndex**, **Supabase** (PostgreSQL + Auth + **RLS**), React + Vite + Tailwind, **Docker**, CI/CD na **GitHub Actions** z deployem landingu na **AWS S3**.

Tracksy — operacyjne SaaS dla salonów beauty 2026 — własny projekt produktowy (2-osobowy zespół)

Problem. Właściciele i managerowie salonów beauty żyją w Booksy, ale platforma nie daje im operacyjnego wglądu w to, co realnie kosztuje pieniądze — puste sloty w grafiku, niewykorzystany czas pracowników, brak szybkich KPI w biegu między klientami.

Rozwiązanie. Nadbudowa nad Booksy łącząca dwa źródła danych — historyczny import raportów XLSX (backfill) oraz near real-time strumień rezerwacji, zmian i odwołań z przychodzących maili przez webhook. Panel zoptymalizowany pod mobile: KPI i alarmy operacyjne czytelne w kilka sekund.

Wartość biznesowa. Szybka reakcja na luki w grafiku przekłada się bezpośrednio na przychód, a osobne adresy e-mail per salon pozwalają obsłużyć wiele lokalizacji bez ingerencji w Booksy.

Stack: Next.js + TypeScript, Supabase (Postgres + Auth + RLS), Supabase Edge Functions jako odbiornik webhooków, Playwright do scrapowania panelu Booksy, monorepo na Turborepo, deploy: Vercel + hosted Supabase.

Stanowisko dydaktyczne Ball on Beam — sterowanie PID/LQR na STM32 2026 — projekt zespołowy, 6. semestr

Problem. Klasyczny niestabilny obiekt sterowania — kulka swobodnie tocząca się po pochylej belce, którą trzeba utrzymać w zadanej pozycji wbrew dynamice układu. Idealna platforma do praktycznej weryfikacji teorii sterowania na realnym sprzęcie.

Rozwiązanie. Kompletny stos mechatroniczny zbudowany od zera w zespole — projekt mechaniczny, identyfikacja modelu obiektu, dwa równoległe regulatory (PID i LQR) z możliwością przełączania w locie oraz desktopowa aplikacja diagnostyczna z telemetrią i strojeniem nastaw w czasie rzeczywistym.

Wartość inżynierska. Porównanie klasycznego sterowania zwrotnego z optymalnym sterowaniem w przestrzeni stanów na tym samym obiekcie. Pełny przepływ: identyfikacja → projekt regulatora → implementacja na MCU → ewaluacja.

Stack: STM32 (C, HAL, FreeRTOS), regulatory PID i LQR, czujnik odległości ToF, serwo sterowane PWM, aplikacja desktopowa w PySide6 + pyqtgraph + OpenCV, identyfikacja modelu w MATLAB/Simulink.

Edukacja

Studia inżynierskie — Automatyka i Robotyka

2023 — obecnie (6. semestr)

Inżynier (w trakcie)

Umiejętności

Języki programowania — Python, TypeScript, C / C++, SQL, MATLAB

AI / LLM (early adopter) — RAG (LlamaIndex), OpenAI API, Claude API, Embeddings + pgvector, routing modeli (LiteLLM), Claude Code / Cursor, MCP

Full-stack web — Next.js, React, FastAPI, TailwindCSS, Turborepo, Playwright

Bazy danych & Cloud — PostgreSQL / Supabase, Row Level Security, Docker, GitHub Actions (CI/CD), AWS S3, Vercel

Embedded & Automatyka — STM32 + FreeRTOS, Arduino, regulatory PID / LQR, MATLAB / Simulink, OpenCV

Języki

Angielski — biegły w dokumentacji technicznej i kontakcie z międzynarodowymi klientami.